

ETL Testing Overview and Automation with Snowflake

1. Overview of Data Migration Testing

- Data Migration Testing ensures data integrity, completeness, and accuracy during system transitions.
- Objectives include validating data integrity, completeness, accuracy, and system compatibility.
- Types: Pre-Migration, Migration, and Post-Migration Testing.
- Common activities: Schema validation, record count matching, data sampling, checksum validation, transformation testing.
- Tools: ETL tools (Informatica, Datastage), SQL, Snowflake, automation scripts.

2. Designing, Developing, and Executing ETL Test Cases

- Understand source-to-target mappings, data models, and transformation logic.
- Design test cases for data validation, transformation logic, data integrity, performance, and reconciliation.
- Develop test cases using SQL and automation tools.
- Execute unit and integration tests, log defects, and re-test.
- Document results and provide stakeholder reports.

3. Python Automation for ETL Testing

- Automate record count validation and row-by-row data comparison.
- Identify mismatches and generate summary reports.
- Extendable to transformation validation, null/duplicate checks, and reporting.

4. Snowflake Adaptation of ETL Test Script

- Use snowflake-connector-python to connect and query Snowflake.
- Fetch data from source and target tables.

- Perform record count and row-level comparisons using pandas.
- Log mismatches and generate reports.

Sample Code Snippet:

```
import snowflake.connector
```

```
import pandas as pd
```

```
conn = snowflake.connector.connect(
```

```
    user='YOUR_USERNAME',
```

```
    password='YOUR_PASSWORD',
```

```
    account='YOUR_ACCOUNT',
```

```
    warehouse='YOUR_WAREHOUSE',
```

```
    database='YOUR_DATABASE',
```

```
    schema='YOUR_SCHEMA'
```

```
)
```

```
def fetch_data(query):
```

```
    cursor = conn.cursor()
```

```
    cursor.execute(query)
```

```
    df = pd.DataFrame(cursor.fetchall(), columns=[col[0] for col in cursor.description])
```

```
    cursor.close()
```

```
    return df
```

```
source_df = fetch_data("SELECT * FROM SOURCE_TABLE")
```

```
target_df = fetch_data("SELECT * FROM TARGET_TABLE")
```

```
def compare_data(source, target):
```

```
mismatches = []  
  
for i in range(len(source)):  
    if not source.iloc[i].equals(target.iloc[i]):  
        mismatches.append((i, source.iloc[i].to_dict(), target.iloc[i].to_dict()))  
  
return mismatches
```

```
mismatches = compare_data(source_df, target_df)  
  
print("Mismatches found:", mismatches)
```